Oral script: the 10-minute defence

Museum Guide project, MSA-DATB09-03. One section per slide, to read aloud.

How to use it

- About 1303 words: at a calm pace (about 130 words per minute), this is about 10 minutes. Without the 111 optional words, about 9 minutes: a safety margin.
- Each slide ends with a link to the next one ("so now we try to fix it"). Keep these links even when you are late: they make the six steps one story.
- Read it aloud 3 times, with the slides, and time yourself once. Speak slowly; pause after each bold sentence: it is the one the jury must remember.
- Blue text in brackets is a stage cue (where to point); do not read it. Grey sentences marked (optional) can be skipped if you are late.
- If you present with a teammate, a natural split is slides 1 to 6, then slides 7 to 11.

Slide 1 · Is a neural network worth it on a museum kiosk?

0:00 to 0:40 · 40 s · 91 words

Good morning. Imagine you are in a museum. You point your phone at a painting, and a guide tells you its style and the story of that movement. Our museum wants exactly this, on visitors' phones and on kiosks in the galleries.

We worked on five styles: Baroque, Cubism, Impressionism, Pop Art and Ukiyo-e. *[point at the five paintings]*

The kiosks have small CPUs, the team is small and the budget is limited. So the director asks one question: **is a neural network worth it in production, or would a simpler classical model do the job?**

Slide 2 · Six steps to answer the director

0:40 to 1:25 · 45 s · 93 words

To answer, we went step by step, and each step told us what to try next. *[point at the six steps, one by one]*

Step one: the data, and a fair test. Step two: our own network, a multi-layer perceptron built from PyTorch primitives. Step three: we watched its training from the inside and learned to diagnose failures. Step four: a pretrained convolutional network, ResNet18. Step five: a final test of every model, classical models included. Step six: the cost on a kiosk CPU.

At the end of these six steps, we will have our answer. Let's start with the data.

Slide 3 · A fair test: one painter, one part

1:25 to 2:20 · 55 s · 119 words

Step one: the data. We took 1 500 paintings from WikiArt, 300 per style. *(optional) We used the course script with a fixed seed, so anyone can download exactly the same images.*

We cut them into three piles: 1 061 to train, 219 to validate, that is, to choose our settings, and 220 to test, used only once, at the very end.

One rule matters a lot: **all the paintings of a known painter go to the same pile.** *[point at the zero]* With a random split, 23 of our 32 known painters would be in two piles, and a model could learn to recognise Monet instead of Impressionism. With our split: zero. **With fair data in place, we could build our first model.**

Slide 4 · Our MLP learns the paintings by heart

2:20 to 3:20 · 60 s · 135 words

Step two. We started with the simplest neural network of the course: a multi-layer perceptron built from nn.Linear layers and ReLU, with our own training loop. It reads the pixels of a 64 by 64 image: 12 288 inputs and 6.4 million parameters.

Before trusting it, we ran two checks. First, a model that knows nothing should have a loss of ln 5, about 1.61. Ours started at 1.99, too confident, so we made its last layer smaller and got 1.59. Second, it must be able to learn one small batch by heart: it does, 100 percent.

Then the real training. *[point at the curves]* The training accuracy goes to 99.7 percent, but the validation accuracy stays around 40 percent. *(optional) And the validation loss goes up after epoch 3.* **This is overfitting: the network learns the paintings by heart.**

Slide 5 · Healthy training, but the MLP overfits

3:20 to 4:25 · 65 s · 136 words

Step three. Our MLP overfits, so we wanted to fix it. But first, we looked inside the network to check that the training itself was healthy: the gradient reaches every layer and only 4 percent of the neurons are dead. So the problem really is overfitting. *(optional) We also tried 8 learning rates and kept 0.0001.*

So we attacked it with the tools of session 6, 3 seeds each. *[point at the chart]* Alone, each tool helps a little. All together, weight decay, dropout, batch norm, flips and early stopping take the MLP from 38.8 to 45.5 percent, and the gap between train and validation from 0.61 to 0.12.

Overfitting is under control. But one number shows a deeper limit: **the MLP finds only 7 percent of the Cubist paintings. It sees colours, not shapes.** This will decide our next model.

Slide 6 · Four broken runs, four signatures

4:25 to 5:20 · 55 s · 123 words

But overfitting is not the only way a training can go wrong. To be ready for other failures, we broke the training four times on purpose, and diagnosed each run. *[point at the table, row by row]*

A learning rate 100 times too big killed 99.6 percent of the neurons of layer 2. A forgotten zero_grad made the gradients pile up: 26 times bigger after two epochs. Raw pixels, without standardisation, gave a first loss of 16 instead of 1.6. And eight sigmoid layers gave a vanishing gradient: the first layer got a millionth of the gradient of the last.

[point at the lesson] The lesson: **two of these bugs show the same symptom, a loss stuck at ln 5. Only the monitoring, dead neurons or the gradient per layer, can tell them apart.**

Slide 7 · A pretrained ResNet18 sees shapes

5:20 to 6:20 · 60 s · 129 words

With a training we can now trust, step four goes back to the deeper limit: our MLP sees colours, not shapes. To see shapes, we need a convolutional network. With only 1 061 images we cannot train one from zero, so we reuse ResNet18, already trained on 1.2 million ImageNet photos.

We freeze it: each painting becomes 512 numbers, and we train only one linear layer, 2 565 weights, in less than a second. *[point at the blue bar]* **Result: 88.1 percent on validation, almost twice our best MLP.** We also fine-tuned the last block: 8.4 million weights for 1 061 images. It overfits: 85.4 percent.

And we measured two silent bugs: forgetting model.eval() drops the score to 12.8 percent, and forgetting the ImageNet normalisation costs 6 points. *(optional) Nothing crashes: only measuring shows them.*

Slide 8 · Seeing the painter inflates the score

6:20 to 7:15 · 55 s · 116 words

But a model this strong can also learn shortcuts, like recognising a painter's hand instead of a style. So we finally tested the painter rule of step one. To measure it fairly, we scored the same images twice with cross-validation, and changed only one thing: whether the model had seen other paintings by the same painter.

[point at the left bars] On images by known painters: 83.7 percent when the painter was seen, 72.3 when not. **11 points of flattery.** *[point at the right bars]* On the control group, the unknown painters, nothing changes.

So the model partly recognises painters, and a random split would have given the director a score that is too good. **Our split protects the result: we can trust the final test.**

Slide 9 · Final test: the pretrained network wins

7:15 to 8:15 · 60 s · 131 words

Step five, the final test: the only time we opened the 220 test paintings, once, for every model, including classical models without any neural network. *[point at the top bars]* The pretrained models are at the top, from 83 to 88 percent. The best classical model, a random forest on hand-made colour and texture features, reaches 67; our MLP 41.

Is the gap real? *[point at the blue card]* A paired bootstrap, 5 000 redraws of the same test paintings, gives our model a lead of **16.4 points over the random forest, between 9 and 24 points: it is not luck.**

[point at the grey card] (optional) One honest note: the fine-tuned model is first on the test, but it was second on validation. With about 220 images, a score moves by about 5 points by chance, so we keep the model we chose before the test.

Slide 10 · Fast and cheap enough for a kiosk

8:15 to 8:55 · 40 s · 90 words

Accuracy is only half of the director's question; the other half is the cost. So step six: is it affordable on a kiosk? We timed one photo at a time, from the image to the answer. *[point at the blue card]* The ResNet needs **53 milliseconds on one CPU thread**, 26 with four threads: faster than a blink. Even on a kiosk four times slower, it answers in a fifth of a second.

The model file is 45 megabytes. *(optional) And when the collection changes, we only recompute the features and retrain one layer, in seconds.*

Slide 11 · Our choice: frozen ResNet18 + linear head

8:55 to 10:00 · 65 s · 140 words

We can now answer the director. *[point at the line under the title]* On the test, our own MLP reached 41 percent, the best classical model 67, and the frozen ResNet18 83, at a cost a kiosk can afford.

So, our answer to the director: yes, a neural network is worth it, but a pretrained one that we reuse. **We recommend a frozen ResNet18 with a linear head.**

[point at the first column] Why? It is accurate: one visitor in six gets a wrong style, against one in three. It is fast enough for a modest CPU, and cheap for a small team: one small layer, retrained in seconds.

[point at the second column] *(optional) We do not recommend our MLP, a classical model alone, or fine-tuning for now.* *[point at the third column]* Before the launch, we must test real photos taken at the kiosks, add more styles, and measure the real hardware.

Thank you. We are happy to take your questions.